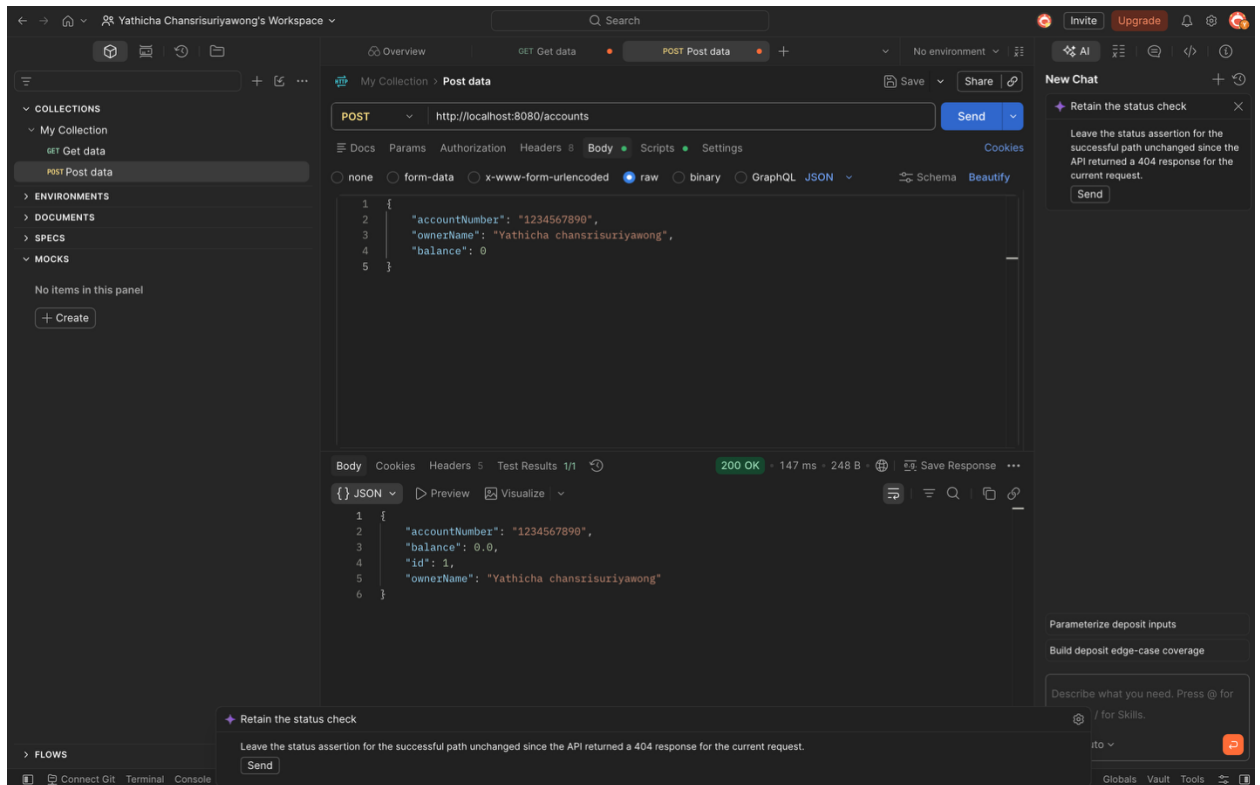


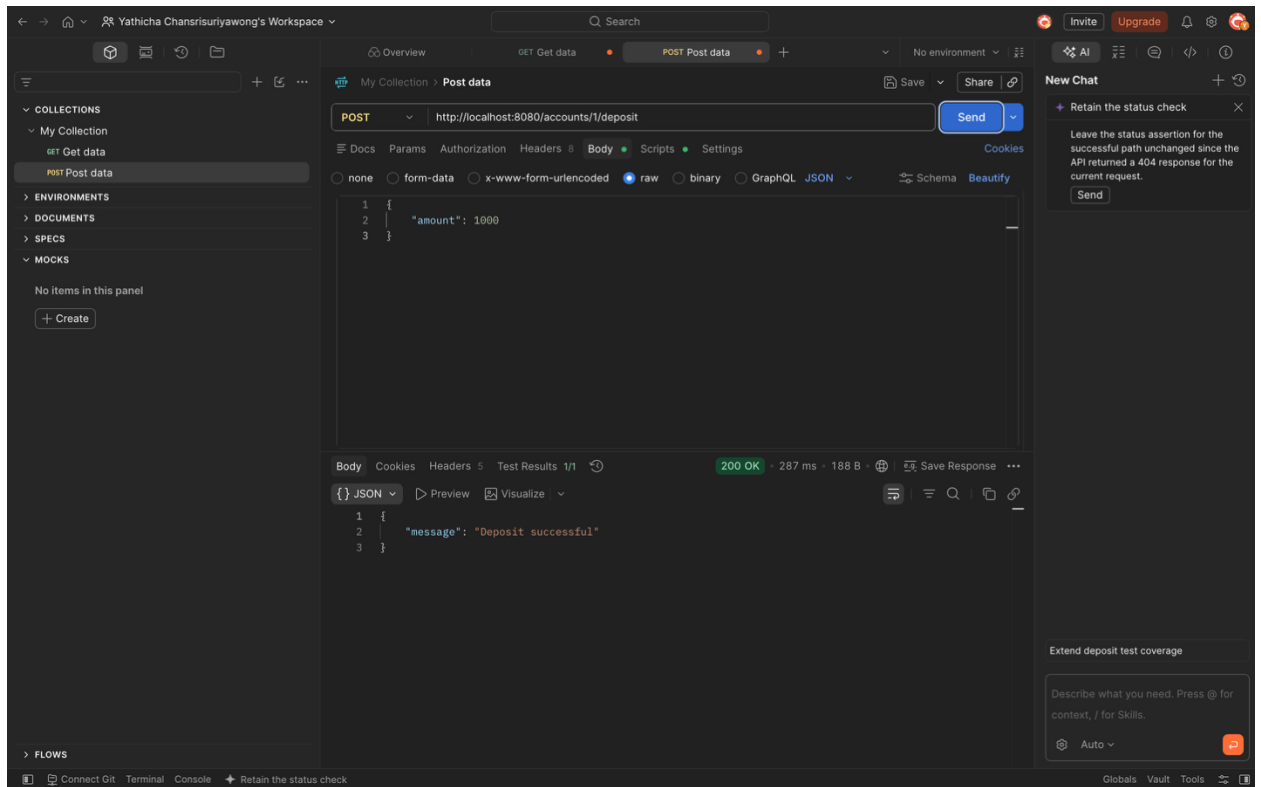
ชื่อ ญาทิชา จันทร์ศรีสุริยวงศ์ รหัสนักศึกษา 673380034-7

1. การสร้าง account



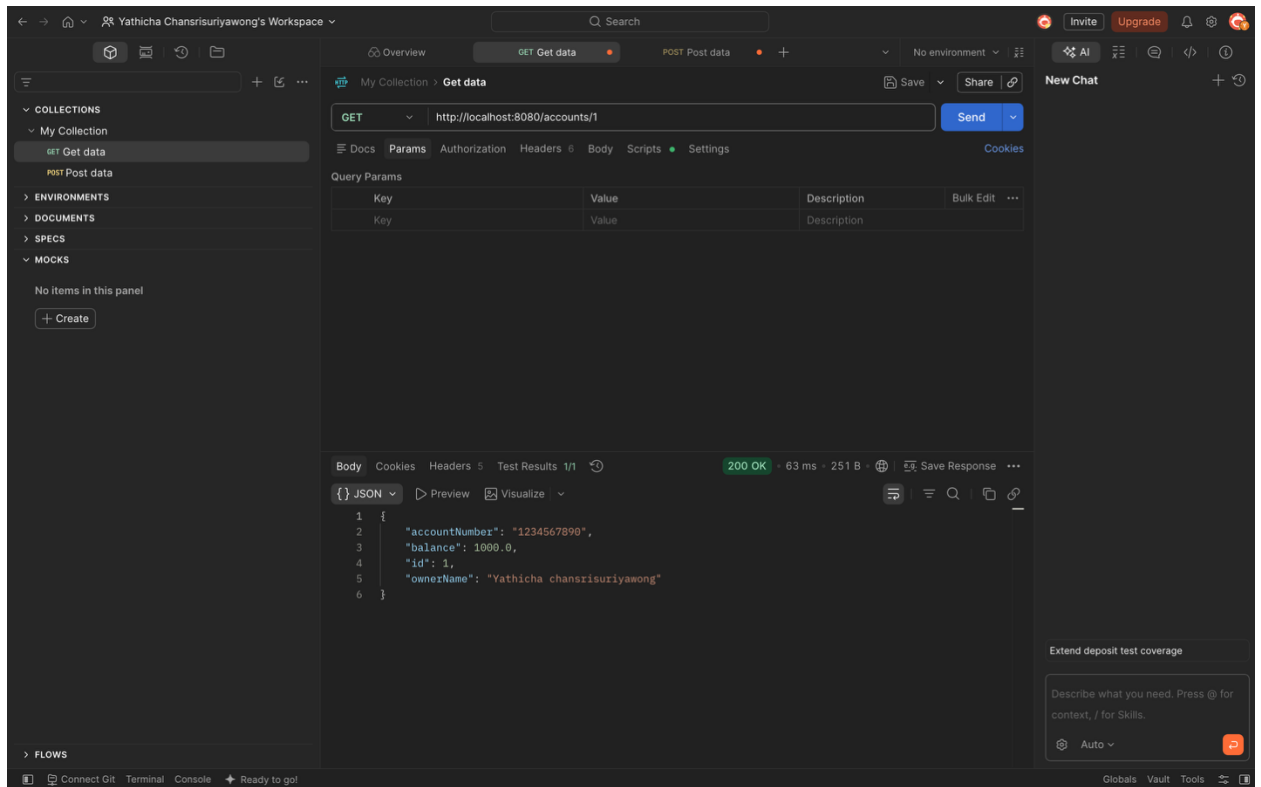
ระบบสามารถสร้าง Account ใหม่ได้สำเร็จ โดยส่ง accountnumber ownername balance ผ่าน HTTP POST ไปยัง /accounts ระบบทำการบันทึกข้อมูลลงในตาราง account ของฐานข้อมูล PostgreSQL และส่งข้อมูล Account ที่สร้างขึ้นกลับมา โดย Account จะได้รับ id จากฐานข้อมูล

2. การฝากเงิน



ส่งจำนวนเงิน 1000 ผ่าน HTTP POST ไปยัง /accounts/{id}/deposit เมื่อทำการฝากเงิน ระบบจะค้นหา Account จาก id จากนั้นเพิ่มจำนวนเงินเข้าไปในยอดคงเหลือ และสร้างข้อมูลการฝากเงินใหม่ใน DepositTransaction หลังจากการทำงานเสร็จสมบูรณ์ Transaction จะถูก Commit ทำให้ข้อมูลทั้ง Account และ DepositTransaction ถูกบันทึกลงในฐานข้อมูล

3. ตรวจสอบ account



ยอดเงินคงเหลือเปลี่ยนจาก 0 เป็น 1000 แสดงว่าการฝากเงินสามารถเพิ่มยอดเงินใน Account
และบันทึกข้อมูลลงในฐานข้อมูลได้สำเร็จ

4. ตรวจสอบ DepositTransaction

The screenshot shows the pgAdmin 4 interface. On the left, the 'Object Explorer' pane displays the database structure, with 'Tables (2)' expanded under the 'public' schema, showing 'account' and 'deposit_transaction'. The 'Query' pane contains the following SQL query:

```
1 SELECT * FROM public.account
2 ORDER BY id ASC
```

The 'Data Output' pane shows the results of the query, displaying a single row with the following data:

id	account_number	balance	owner_name
1	1234567890	1000	Yathicha chansrisuriyawo...

The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.082'.

จากการตรวจสอบตาราง deposit_transaction พบว่ามีข้อมูลรายการฝากเงินจำนวน 1000 ถูกบันทึกไว้

5. ทดลอง Rollback

Yathicha Chansrisuriyawong's Workspace

Overview GET Get data POST Post data No environment

My Collection > Post data

POST http://localhost:8080/accounts/1/deposit

Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "amount": 1000
3 }
```

Body Cookies Headers 4 Test Results 0/1 500 Internal Server Error 272 ms · 5.68 KB Save Response

JSON Preview Debug with AI

```
1 {
2   "timestamp": "2026-09-15T12:42:19.525Z",
3   "status": 500,
4   "error": "Internal Server Error",
5   "trace": "java.lang.RuntimeException: Test Rollback\n\tat com.example.demo.service.DepositService.deposit(DepositService.java:35)\n\tat java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor.java:104)\n\tat java.base/java.lang.reflect.Method.invoke(Method.java:565)\n\tat org.springframework.aop.support.AopUtils.invokeJoinpointUsingReflection(AopUtils.java:359)\n\tat org.springframework.aop.framework.ReflectiveMethodInvocation.invokeJoinpoint(ReflectiveMethodInvocation.java:190)\n\tat org.springframework.aop.framework.ReflectiveMethodInvocation.proceed(ReflectiveMethodInvocation.java:158)\n\tat org.springframework.transaction.interceptor.TransactionInterceptor$1.proceedWithInvocation(TransactionInterceptor.java:133)\n\tat org.springframework.transaction.interceptor.TransactionAspectSupport.invokeWithinTransaction(TransactionAspectSupport.java:371)\n\tat org.springframework.transaction.interceptor.TransactionInterceptor.invoke(TransactionInterceptor.java:130)\n\tat org.springframework.aop.framework.ReflectiveMethodInvocation.proceed(ReflectiveMethodInvocation.java:170)\n\tat "
```

Flows

Connect Git Terminal Console Test the error response structure

Invited Upgrade

New Chat

Test the error response structure

Add assertions to validate the 500 Internal Server Error response structure, checking the timestamp, status code, and error fields.

Test script

```
1 1 pm.test("Successful POST request", function () {
2 2   pm.expect(pm.response.status).toBe(500);
3 3   pm.expect(pm.response.json().status).toBe(500);
4 4   pm.expect(pm.response.json().error).toBe("Internal Server Error");
5 5   pm.test("Response structure is correct", function () {
6 6     pm.expect(pm.response.json().timestamp).toBeTruthy();
7 7     pm.expect(pm.response.json().trace).toBeTruthy();
8 8   });
9 9 });
```

Apply

Parameterize deposit inputs

Standardize the local API setup

Describe what you need. Press @ for context, / for Skills.

Auto

Yathicha Chansrisuriyawong's Workspace

Overview GET Get data POST Post data No environment

My Collection > Get data

GET http://localhost:8080/accounts/1

Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 5 Test Results 1/1 200 OK 59 ms · 251 B Save Response

JSON Preview Visualize

```
1 {
2   "accountNumber": "1234567890",
3   "balance": 1000.0,
4   "id": 1,
5   "ownerName": "Yathicha chansrisuriyawong"
6 }
```

Flows

Connect Git Terminal Console Ready to go

Invited Upgrade

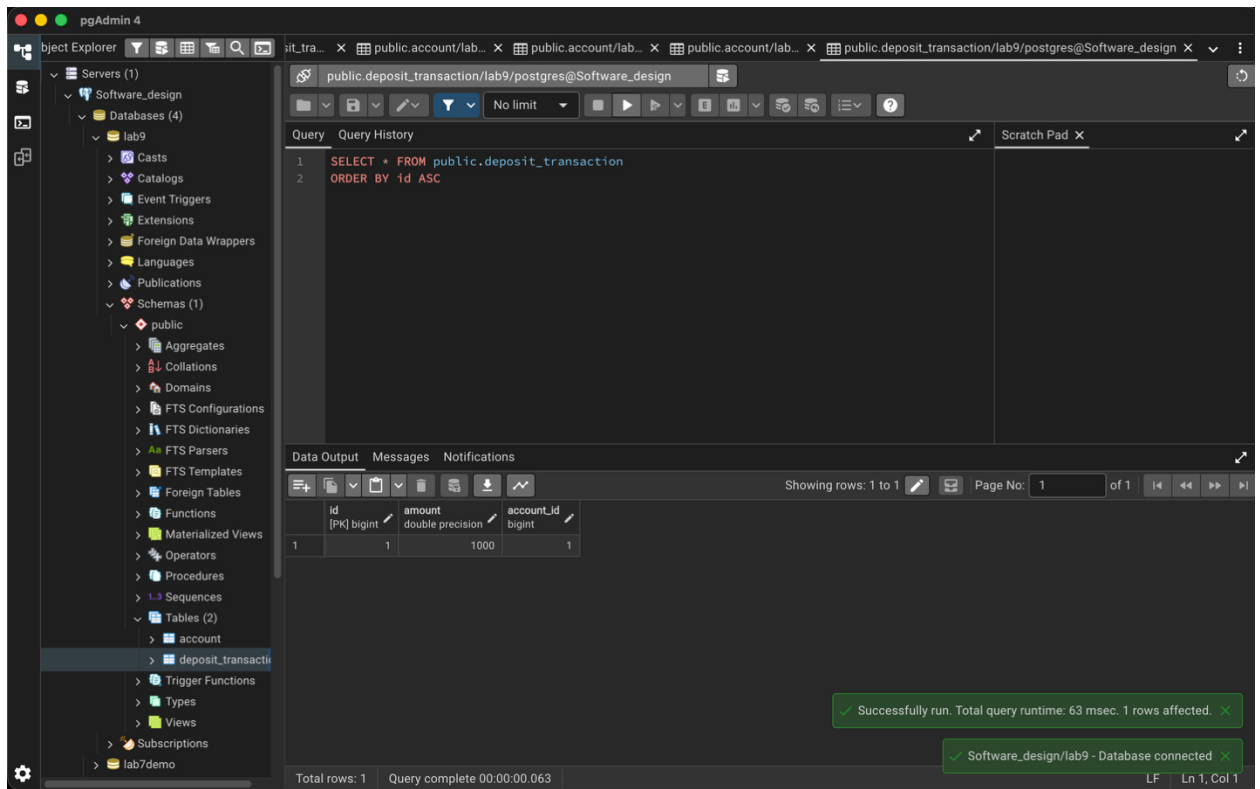
New Chat

Parameterize deposit inputs

Standardize the local API setup

Describe what you need. Press @ for context, / for Skills.

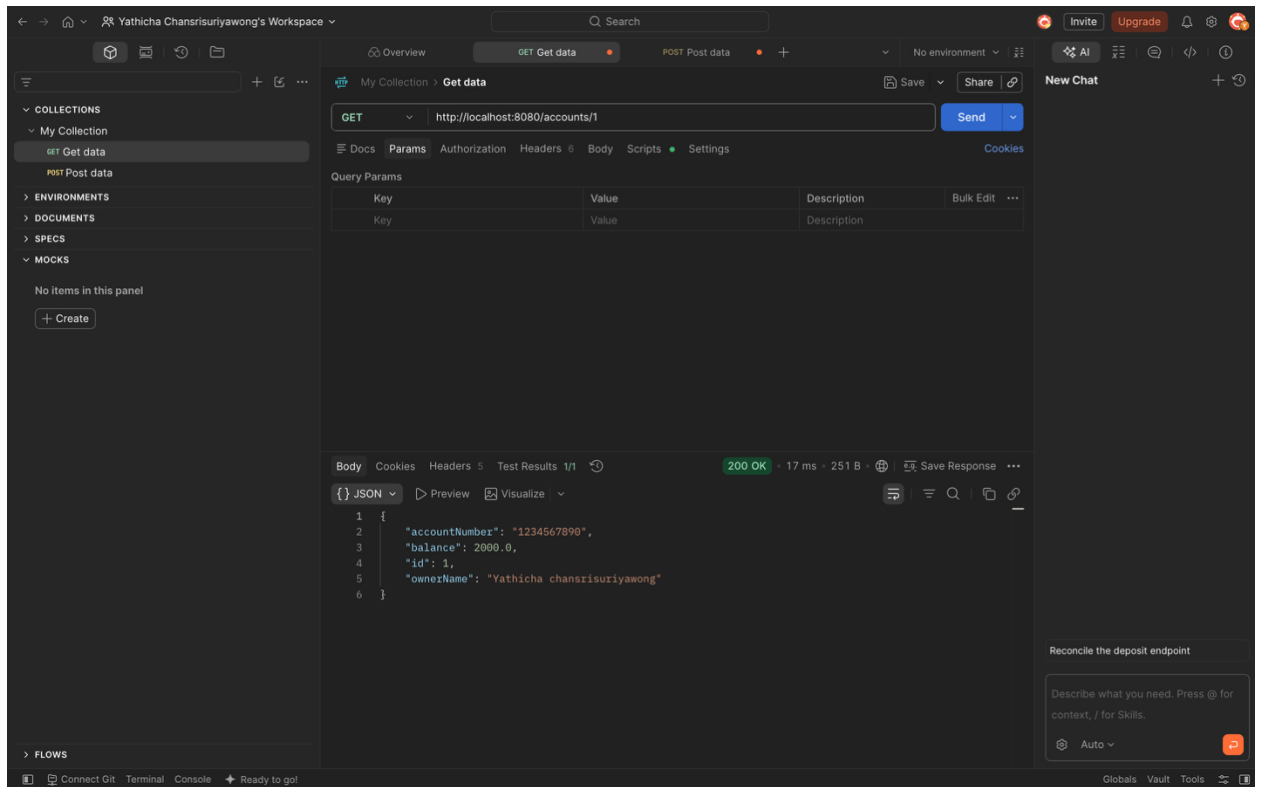
Auto



เพิ่ม `throw new RuntimeException("Test Rollback");` หลังจากที่เราทำการบันทึก Account และ DepositTransaction แล้ว เมื่อเกิด Exception ระบบพบว่าการทำงานของ Transaction ไม่สำเร็จ จึงทำการ Rollback การเปลี่ยนแปลงทั้งหมดภายใน Transaction เนื่องจาก method `deposit()` มี `@Transactional`

ผลที่ได้คือยอดเงินของ Account ยังคงเป็นค่าก่อนการฝากเงิน และไม่มี DepositTransaction รายการใหม่ถูกบันทึกลงในฐานข้อมูล

6. เมื่อไม่มี @Transactional



การเปลี่ยนแปลงที่ถูก Commit ไปแล้วอาจยังคงอยู่ในฐานข้อมูล ส่งผลให้ Account อาจมี ยอดเงินเพิ่มขึ้น และ DepositTransaction อาจยังมีรายการฝากเงิน แม้ว่าการทำงานโดยรวมจะจบด้วย Error ก็ตาม

คำถามท้ายแลป

1. @Transactional มีหน้าที่อะไร?

@Transactional ใช้สำหรับกำหนดขอบเขตของ Transaction ให้กับ method ที่ระบุ โดยคำสั่งที่เกี่ยวข้องกับฐานข้อมูลภายใน method จะถูกจัดการเป็น Transaction เดียวกัน หากการทำงานทั้งหมดสำเร็จ Transaction จะถูก Commit แต่หากเกิด Exception ระบบสามารถ Rollback การเปลี่ยนแปลงภายใน Transaction ได้

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

การฝากเงินมีการเปลี่ยนแปลงข้อมูลมากกว่าหนึ่งส่วน หากขั้นตอนใดขั้นตอนหนึ่งเกิดข้อผิดพลาด ระบบสามารถ Rollback เพื่อป้องกันไม่ให้เกิดข้อมูลที่ไม่สอดคล้องกัน

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

- COMMIT คือการยืนยันการเปลี่ยนแปลงที่เกิดขึ้นภายใน Transaction
- ROLLBACK คือการยกเลิกการเปลี่ยนแปลงที่เกิดขึ้นภายใน Transaction

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

DepositTransaction มีความสัมพันธ์แบบ ManyToOne กับ Account หนึ่งบัญชีสามารถมีรายการฝากเงินได้หลายรายการ

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

หลังจาก Save Account แต่ยังไม่อยู่ใน Transaction เดียวกันกับการสร้าง DepositTransaction ระบบจะทำการ Rollback Transaction ทำให้การเปลี่ยนแปลงของ Account ถูกยกเลิกด้วย

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

หาก accountRepository.save() สำเร็จ แต่เกิด Exception ก่อนหรือหลังการบันทึก DepositTransaction การเปลี่ยนแปลงที่ถูก Commit ไปแล้วอาจไม่สามารถ Rollback กลับมาพร้อมกันได้